

torch.fft.rfft#

<https://pytorch.org/docs/stable/generated/torch.fft.rfft.html>

Description:

Computes the one dimensional Fourier transform of real-valued input. The FFT of a real signal is Hermitian-symmetric, $X[i] = \text{conj}(X[-i])$ so the output contains only the positive frequencies below the Nyquist frequency. To compute the full output, use `fft()` Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. input (Tensor) – the real input tensor

Function Signature

```
torch.fft.rfft(input, n=None, dim=-1, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.arange(4)
>>> t
tensor([0, 1, 2, 3])
>>> torch.fft.rfft(t)
tensor([ 6.+0.j, -2.+2.j, -2.+0.j])
```

```
>>> torch.fft.fft(t)
tensor([ 6.+0.j, -2.+2.j, -2.+0.j, -2.-2.j])
```

Notes & Warnings

NOTE Note Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

Detailed Documentation

Documentation

Computes the one dimensional Fourier transform of real-valued input.

The FFT of a real signal is Hermitian-symmetric, $X[i] = \text{conj}(X[-i])$ so the output contains only the positive frequencies below the Nyquist frequency. To compute the full output, use `fft()`

Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension.

`input (Tensor)` – the real input tensor

`n (int, optional)` – Signal length. If given, the input will either be zero-padded or trimmed to this length before computing the real FFT.

`dim (int, optional)` – The dimension along which to take the one dimensional real FFT.

`norm (str, optional)` –

Normalization mode. For the forward transform (`rfft()`), these correspond to:

"forward" - normalize by $1/n$

"backward" - no normalization

"ortho" - normalize by $1/\text{sqrt}(n)$ (making the FFT orthonormal)

Calling the backward transform (`irfft()`) with the same normalization mode will apply an

overall normalization of $1/n$ between the two transforms. This is required to make `irfft()` the exact inverse.

Default is "backward" (no normalization).

`out` (Tensor, optional) – the output tensor.

Compare against the full output from `fft()`:

Notice that the symmetric element $T[-1] == T[1].conj()$ is omitted. At the Nyquist frequency $T[-2] == T[2]$ is it's own symmetric pair, and therefore must always be real-valued.